```
five(*['a', 'b'], 1, 2, 3) # => "I was passed a b 1 2 3"
five(*(10..14))            # => "I was passed 10 11 12 13 14"
five(*[1,2], 3, *(4..5))   # => "I was passed 1 2 3 4 5"
```

Splat arguments can appear anywhere in the argument list, and you can intermix splat and regular arguments.

Similarly, you can expand hashes, or anything that implements to_h, into keyword arguments by prefixing the argument with a double-splat:

```
def method_with_keywords(city:, state:, zip:)
  "I live in #{city}, #{state} #{zip}"
end

data = {city: "Chicago", state: "IL", zip: "60606"}
puts method_with_keywords(**data)
```

produces:

```
I live in Chicago, IL 60606
```

You can also use the shortcut access syntax if the name of the keyword and the name of the variable in the local context are the same, similar to the Hash shortcut we saw in Hashes, on page 56.

```
def method_with_keywords(city:, state:, zip:)
  "I live in #{city}, #{state} #{zip}"
end

city = "Chicago"
state = "IL"
zip = "60606"
puts method_with_keywords(city:, state:, zip:)
```

produces:

```
I live in Chicago, IL 60606
```

Passing Block Arguments

Earlier we saw how an & in a parameter list converted a block argument to a Proc object. You can also do this in reverse by passing a Proc object, or anything that implements the method to_proc, and prefixing it with an & to convert it to a block argument.

A very common example of the use of objects that implement to_proc is Symbol. The following two lines of code behave identically:

```
["a", "b", "c"].map { |s| s.upcase } # => ["A", "B", "C"]
["a", "b", "c"].map(&:upcase)        # => ["A", "B", "C"]
```

And the reason why this works is that the class Symbol implements the to_proc method, returning a Proc object that says "take the argument to this proc, and call the method whose name matches this symbol". The returned Proc object gets used as the block argument and behaves the same as the explicit block in the first line. You'll frequently see this syntax as a shortcut for methods that take simple blocks like map or sort_by.

We've already seen how to associate a block with a method call:

```
collection.each do |member|
  # ...
```